Programación Avanzada - Sección 1

Ayudantía 7: Listas, Ordenamiento y Binary Search

Profesor: Vicente Rivera

Ayudantes: Diego Banda, Martin Ramos



Contacto

Ayudante Cátedra

diego.banda@mail.udp.cl

darklouds

Click aquí!!

Ayudante Corrector

martin.ramos1@mail.udp.cl

martinxikito



ArrayList

Un arreglo dinámico, crece automáticamente.

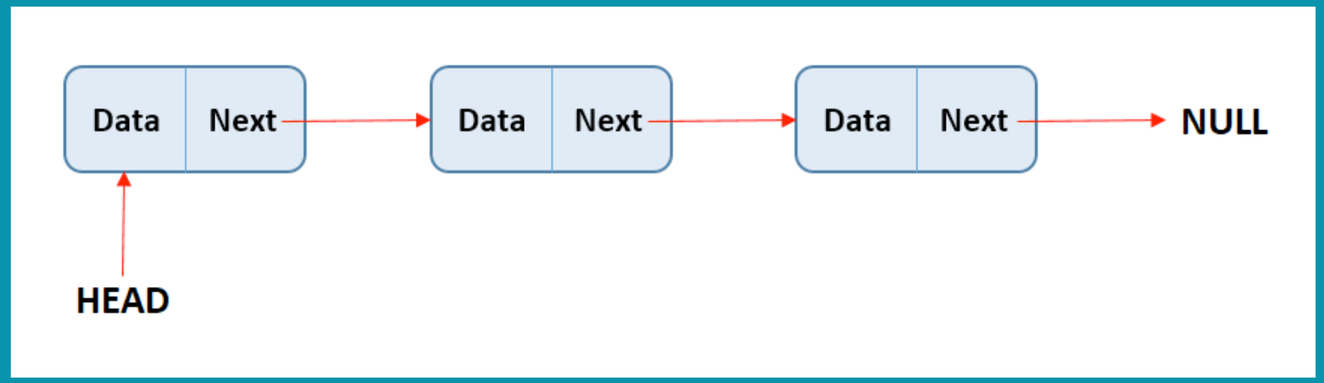
- **Tamaño:** Elementos actuales en el *ArrayList*.
- **Capacidad:** Cuantos elementos caben antes de tener que aumentar su capacidad (lo hace automáticamente)

Cuando se supera la Capacidad, automáticamente duplica su tamaño y así puede seguir agregando datos.

Operación	Qué hace	Operación	Qué hace
Add(X)	Inserta el dato al final de la lista	set(i, X)	Reemplaza el valor de la posición i con X
add(i, X)	Inserta el dato en la posición i, desplaza el resto hacia la derecha	remove(i)	Quita el elemento de la posición i, desplazando el resto hacia la izquierda
get(i)	Obtiene el valor de la posición i	size()	Retorna el tamaño actual

LinkedList

La *linkedList* es una estructura de datos altamente utilizada, basada en referencias, siempre se tiene un dato y un nodo Next.




```
1   class node {
2       int data;
3       node next;
4       public node(int data, node next){
5           this.data = data;
6           this.next = next;
7       }
8       public node(int data){
9           this.data = data;
10          this.next = NULL;
11      }
12  }
13
14  public class linkedlist {
15      node head;
16      public linkedlist(){
17          head = NULL;
18      }
19  }
```

LinkedList

Se utiliza debido a su habilidad para realizar algunas inserciones rápidamente y flexibilidad ante algunos casos, pero principalmente es **la base** de otras estructuras de datos.

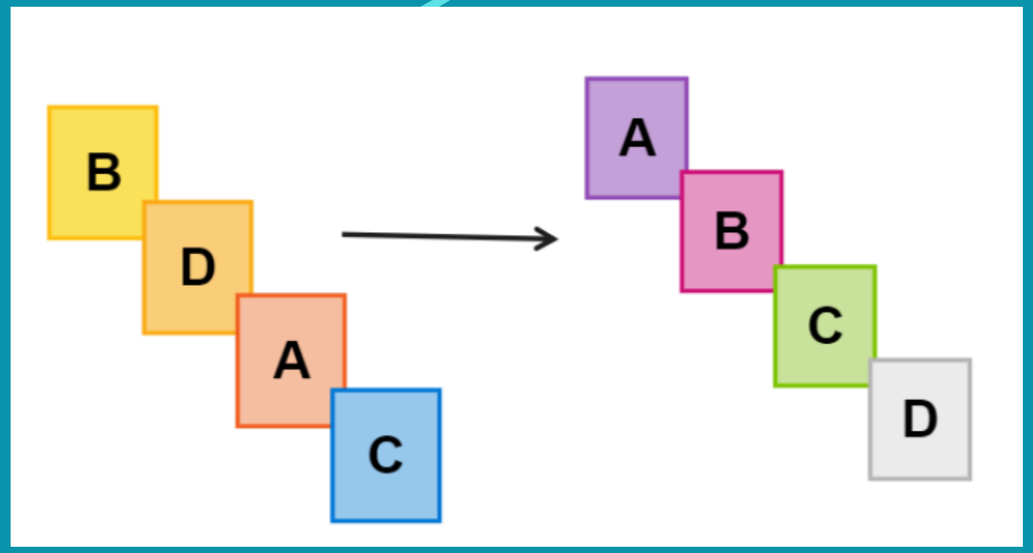
Algunos métodos (que idealmente programen) son:

- **insertFirst:** Para insertar al inicio.
- **insertLast:** Para insertar al final.
- **insertAt(i):** Insertar en la posición i.
- **at(i):** Buscar valor en posición i.
- **removeFirst():** Elimina el primero.
- **removeLast():** Elimina el último.
- **removeAt(i):** Elimina el elemento en la posición i.
- **removeValue(N):** Elimina el nodo de valor N.
- Y una larga lista...



Ordenamiento

Las técnicas de ordenamiento nos sirven mucho en la computación, principalmente como un paso de preprocesado para luego ejecutar acciones rápidamente.



Concepto importante: Estabilidad

Un algoritmo de ordenamiento se considera estable si los elementos repetidos quedan en el mismo orden entre sí.

Ordenamiento: Bubble Sort

Bubble sort ordena interativamente comparando cada elemento del arreglo con el de su derecha, dejando los menores a la izquierda y los mayores a la derecha.

- **Es estable.**

```java
public static void bubbleSort(int[] arr){
    int temp;
    int N = arr.length;
    for(int i = 0 ; i < N - 1 ; i++){
        for(int j = 0 ; j < N - 1 - i ; j++){
            if(arr[j] > arr[j+1]){
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

Ordenamiento: Insertion Sort

Insertion Sort mantiene la parte de la izquierda ordenada, selecciona de la derecha el siguiente número y lo agrega en la posición que mantenga el orden.

```java
public static void insertionSort(int[] arr){
    int N = arr.length;
    int key;
    for(int i = 1 ; i < N ; i++){
        key = arr[i];
        int j = i - 1;
        while(j >= 0 && arr[j] > key){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

Binary Search

Binary search es la justificación de ordenar un arreglo antes, podemos búscar de manera muchísimo más eficiente

```java
public static int binarySearch(int[] arr, int target){ //  Actuamente funciona solo para números enteros
    int l = 0;
    int r = arr.length - 1;
    while(l <= r){
        mid = (l + r)/2;
        if(arr[mid] == target){
            return mid;
        }else if(arr[mid] < target){
            l = mid + 1;
        }else{
            r = mid - 1;
        }
    }
    return -1;  //  Se identifica -1 como no encontrado
}
```

Ejercicios

1. Dado un ArrayList<String> imprimir todos los elementos que tengan menos de 5 elementos junto a su indice.
2. Eliminar todos los elementos duplicados de un arraylist manteniendo su orden de aparición
3. Dado un ArrayList<int> y un rango se rota solo el sub arreglo dado. Ejemplo: arr: 1, 2, 6, 5, 10 y rango: 2, 4 resultado: 1, 2, 10, 5, 6.
4. Implementar LinkedList con funciones: Insertar final, insertar inicio, eliminar indice, eliminar valor.
5. Implementar bubble sort e insertion sort, pero para ordenar de mayor a menor.
6. Dado un arreglo ordenado, contar cuantas veces aparece un número búscado.
7. Implementar una clase jugador con id, nombre y puntajeActual, luego crear un arreglo que ordene de menor a mayor por puntaje del jugador, y finalmente dado un target de puntaje buscar cuantos jugadores hay con tal puntaje.





Muchas gracias!

